INSTITUTO SUPERIOR POLITÉCNICO CÓRDOBA

Tecnicatura:
TECNICATURA SUPERIOR EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL



INFORME COMPLETO - EVIDENCIA III - ABP

|MÓDULO - Práctica PROFESIONALIZANTE I

|Docente - ACCIETTO Dianela

|Grupo - N²-EFMG Solutions

[github -NN-EFMG-Solutions](#) Videos y Photos de PRUEBAS

Cliente: Escuela PRoA (Río Tercero)	**Desarrollador: N²- EFMG**
	Integrantes & Roles:
Fecha: Septiembre de 2026	• Valeria Nieves Villalba (Business Analyst) • Florencia Santarena (Data Scientist / Backend) • Mariela Suarez (Machine Learning Engineer) • Juan Gabriel Cabrera (Full Stack QA Engineer) • Nicolás Allende Olmedo (Project Manager) • Emanuel Guaraz (IA Engineer)

INTRODUCCIÓN Y CONTEXTO DEL PROYECTO

El presente informe técnico detalla la evolución de la solución desarrollada por la empresa **N²-EFMG Solutions** para el **Sector Rural de Río Tercero**. En la fase inicial del Proyecto Integrador, se identificó un punto de dolor operativo crítico: los Sectores Rurales y Pequeños Productores de la comunidad y los entes locales carecían de una infraestructura centralizada y automatizada para almacenar e historiar las variables climáticas y ambientales, que puedan obtener y analizar de manera sencilla a los fines de tomar mejores decisiones productivas, con la finalidad de ampliar sus márgenes de ganancias, y prever pérdidas por desastres climáticos. Esto obliga a los usuarios a consultar datos generalizados de forma manual,y sin ningún tipo de personalización, a centros de datos climáticos de la zona, los cuales resultan de difícil acceso para el común de las personas, y datos crudos sin ningún tipo de análisis, reportes o informes, del cual puedan obtener información precisa para una toma de decisiones empresariales reales, aplicables al sector productivo que las consulte.

Para mitigar este problema, se implementó exitosamente una canalización de ingesta robusta mediante un script en Python que captura los datos del puerto serie vía USB y los persiste de manera estructurada en una base de datos MySQL local (con soporte de un buffer offline en formato CSV ante caídas de conexión). No obstante, para que esta valiosa base de datos se traduzca en decisiones inteligentes y accesibles en tiempo real —tanto para directivos, docentes y estudiantes, como para entes municipales e industriales que sufren la falta de microdatos meteorológicos y de calidad ambiental precisos en el departamento Tercero Arriba—, se requiere una interfaz de usuario accesible y sin fricción. Es así como surge el diseño de una aplicación capaz de consultar la **Base de Datos de la Estación Meteorológica** en tiempo real, mediante el desarrollo de una **API Rest**, y que los muestre en una **Interfaz Web** intuitiva, de fácil lectura y dinámica, para una visualización concreta del estado ambiental local. Y a futuro, la implementación de un **Chatbot de WhatsApp**, orientado a la consulta inmediata y democratización de los datos ambientales en Río Tercero y la región.

01. DOCUMENTO DE REQUERIMIENTOS Y TRAZABILIDAD

De acuerdo con los lineamientos establecidos, cada requerimiento funcional está trazado formalmente a los puntos de dolor (Pain Points) detectados originalmente. A continuación, se presenta la matriz de requerimientos del sistema de visualización y alertas a través de Telegram.

ID	Pain Point	Requerimiento Técnico Funcional (RF) / No Funcional (RNF)	Descripción del Requerimiento
RF-06	PP-04 Experiencia (Falta de Visibilidad)	**Consulta de Clima en Tiempo Real**	La API Rest debe consultar de manera instantánea la Base de Datos cada 20 minutos, retornando los últimos registros de Temperatura, Humedad, Calidad del Aire y Probabilidad de Lluvia, y visualizarlas en la Interfaz Web, junto con la fecha y hora del sensor.
RF - 07	PP-04 Experiencia (Falta de Visibilidad)	**Geolocalización**	La aplicación debe mostrar, mediante Geolocalización, la ubicación en tiempo real del usuario, o la zona donde se encuentre el Dispositivo de la Estación Meteorológica.
RF-02	PP-03 Reportes (Falta de Reportes Personalizados)	**Análisis de Tendencia (Estado del Tiempo)**	La aplicación debe visualizar las variaciones de las variables de Temperatura, Humedad y Probabilidad de Lluvia, mediante gráficos interactivos, de las últimas 24 hs y de la última semana.

RF-07	PP-04 Experiencia (Falta de Visibilización)	**Visualización de Pronóstico Semanal (Estado del Tiempo)**	La aplicación debe mostrar una simulación del clima, para los próximos 6 días, mostrando Temperatura, Humedad y Probabilidad de Lluvia.
RF-07	PP-04 Experiencia (Falta de Visibilización)	**Visualización de Calidad del Aire (Reporte de Gases Nocivos)**	La aplicación debe visualizar, mediante el rótulo Calidad del Aire, la existencia o no, o la nocividad de Gases Tóxicos en el entorno o zona donde se encuentre el Usuario. de 0 a 160 Óptimo, de 161 a 190 Alerta y de =< 191 Peligro.
RF-03	PP-02 Datos (Datos dispersos/sucios)	**Consolidación de Reportes Diario**	La aplicación debe disponer de la posibilidad de generar Reportes en formato PDF o Excel, sobre los registros obtenidos, para un análisis exhaustivo y profesional, si el usuario así lo desea
RNF-01	PP-04 Experiencia (Visibilidad)	**Latencia de Alertas Corta (<20 minutos)**	El tiempo transcurrido desde que un sensor Arduino escribe una temperatura fuera de rango en la base de datos MySQL hasta el envío del mensaje de alerta en WhatsApp no debe superar los 20 minutos.
Desarrollo de Alertas Tempranas: (Para Práctica Profesionalizante II)			
RNF-01	PP-04 Experiencia (Falta de Visibilidad)	**Consulta de Clima en Tiempo Real**	El Chatbot debe responder de manera instantánea ante las palabras clave 'Clima' o 'Actual', retornando el último registro de temperatura persistido en MySQL, junto con la fecha y hora del sensor.
RF-02	PP-03 Decisión (Decisión sin Sustento)	**Análisis de Tendencia (Estado del Tiempo)**	El Chatbot debe responder a la consulta 'Estado del Tiempo' calculando la pendiente de temperatura de la última hora en la base de datos, informando si el clima se encuentra en aumento, estable o en descenso.
RF-03	PP-02 Datos (Datos dispersos/sucios)	**Consolidación de 'Reporte' Diario**	Al recibir la palabra 'Reporte', el Chatbot ejecutará un análisis agregativo en MySQL (utilizando Pandas en backend) y devolverá las temperaturas máxima, mínima y promedio de las últimas 24 horas.
RNF-01	PP-04 Experiencia (Visibilidad)	**Latencia de Alertas Corta (<60 segundos)**	El tiempo transcurrido desde que un sensor Arduino escribe una temperatura fuera de rango en la base de datos MySQL hasta el envío del mensaje de alerta en WhatsApp no debe superar los 60 segundos.
RNF-02	PP-01 Proceso (Fricción Operativa)	**Disponibilidad Continua (24/7)**	La API Webhook del Chatbot y el daemon de alertas deben ejecutarse de manera ininterrumpida con un uptime objetivo del 99.5%.
RF-04	PP-04 Experiencia / PP-01 (Fricción y Alertas)	**Alerta de Temperatura Crítica Alta (>32°C)**	Un servicio demonio en Python debe escanear la base de datos. Si la temperatura ingresada es superior a 32°C, se disparará automáticamente una alerta masiva al grupo de WhatsApp establecido.

RF-05	PP-04 Experiencia / PP-01 (Fricción y Alertas)	**Alerta de Temperatura Crítica Baja (<8°C)**	Si el escaneo de la base de datos registra una lectura de temperatura inferior a 8°C, el demonio emitirá una notificación de alerta crítica de helada o frío extremo al canal de WhatsApp.

Definición de la Solución de Fase 2: El Chatbot de WhatsApp actuará como una interfaz interactiva de consulta. Cualquier usuario autorizado podrá enviar mensajes predefinidos para obtener de forma instantánea el 'Clima' (lectura en tiempo real), el 'Estado del Tiempo' (tendencia reciente) o un 'Reporte' consolidado del día. Además, el sistema integrará un servicio demonio proactivo encargado de lanzar alertas críticas inmediatas vía WhatsApp cuando la temperatura registrada en la base de datos supere los 32°C o sea inferior a 8°C, salvaguardando así las actividades del aula e invernaderos.

CRITERIOS DE ACEPTACIÓN

Para validar el correcto funcionamiento de la API REST y de la Interfaz de Usuario, se definen los siguientes escenarios de prueba y criterios de aceptación específicos:

Escenario: Consulta de Clima Actual (RF-01)	
Dado que...	La base de datos MySQL tiene un últimos registros persistidos de Temperatura, Humedad y Calidad del Aire
Cuando...	La API Rest, consulta los datos solicitados al Base de Datos de My SQL denominada **estacionmeteorologica_proa**
Entonces...	La Interfaz de Usuario, muestra los registros consultados en Tiempo Real, para su valoración y estudio.

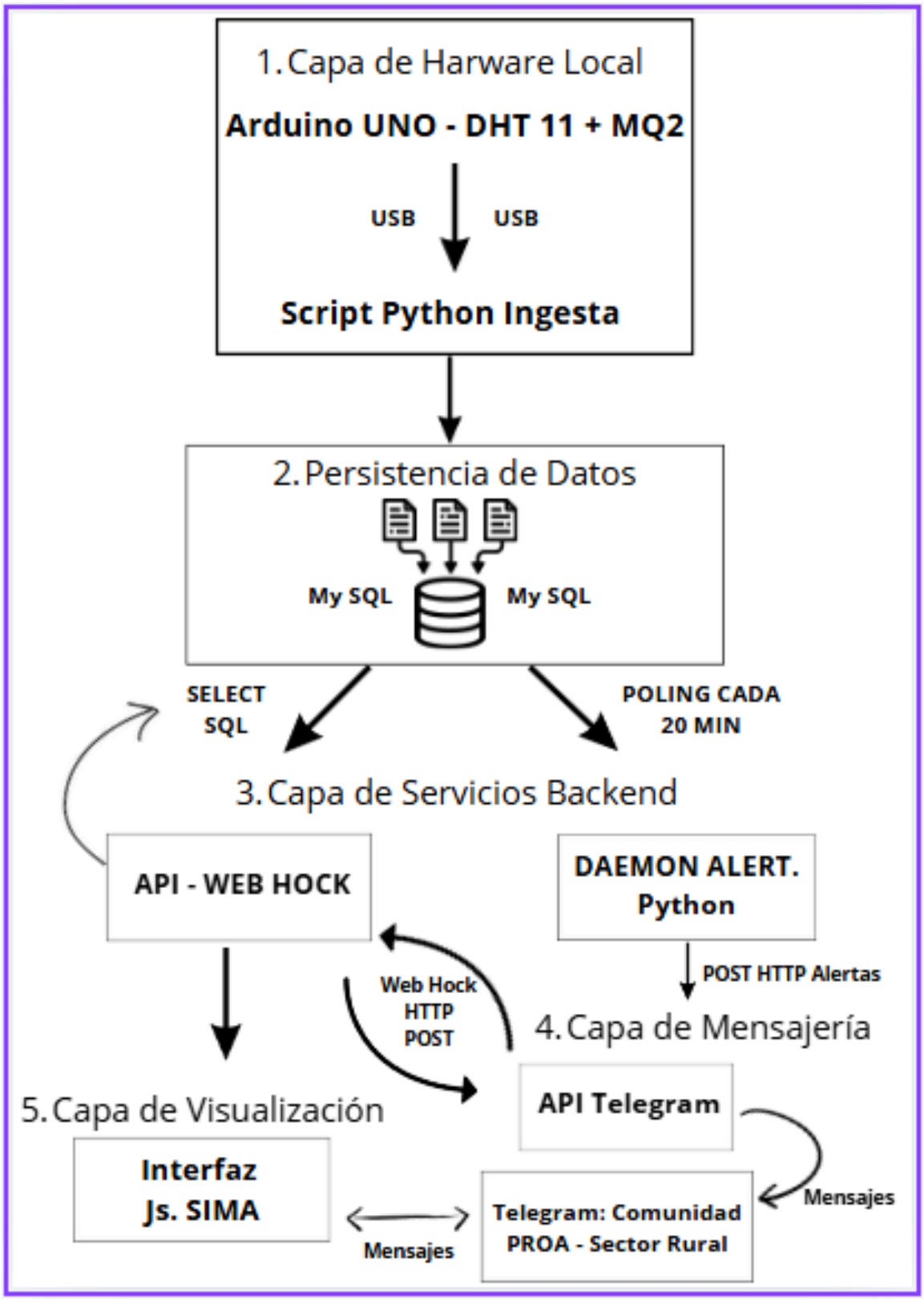
Desarrollo de Alertas Tempranas: (Para Práctica Profesionalizante II)

Escenario: Alerta Proactiva de Temperatura Crítica Alta (RF-04)	
Dado que...	El sensor Arduino registra un pico de calor en el invernadero de 32.5°C y lo inserta automáticamente en la tabla MySQL.
Cuando...	El demonio de monitoreo de alertas realiza su escaneo periódico de 10 segundos y detecta un valor superior a 32°C.
Entonces...	El demonio envía una petición POST HTTP a Twilio, la cual redirige en menos de 10 segundos una notificación crítica a los contactos del colegio PRoA diciendo: *' ALERTA DE CALOR: Temperatura crítica superada en Escuela PRoA Río Tercero: 32.5°C. Tomar precauciones de ventilación.'* El sistema bloquea alertas repetitivas por los próximos 30 minutos.

Escenario: Alerta Proactiva de Temperatura Crítica Baja (RF-05)	
Dado que...	La temperatura del entorno desciende a 7.5°C debido a una helada invernal nocturna, registrándose en MySQL.
Cuando...	El servicio demonio detecta en su lectura recurrente que el valor es inferior a 8°C.
Entonces...	El demonio dispara de inmediato una notificación push de WhatsApp indicando: *' ALERTA DE HELADA: Temperatura crítica baja en Escuela PRoA Río Tercero: 7.5°C. Activar protocolos de protección térmica.'*

02. ARQUITECTURA DEL SISTEMA Y FLUJO DE DATOS

El esqueleto técnico de la solución se ha estructuradoutilizando un acoplamiento laxo entre los módulos para garantizar escalabilidad, facilidad de mantenimiento y desacoplamiento de la base de datos.


1. Capa de Harware Local
Arduino UNO - DHT 11 + MQ2
USB
USB
Script Python Ingesta
2. Persistencia de Datos
My SQL
My SQL
SELECT
SQL
POLING CADA
20 MIN
3. Capa de Servicios Backend
API - WEB HOCK
DAEMON ALERT.
Python
POST HTTP Alertas
Web Hock
HTTP
POST
4. Capa de Mensajería
5. Capa de Visualización
API Telegram
Interfaz
Js. SIMA
Mensajes
Telegram: Comunidad
PROA - Sector Rural
Mensajes

EXPLICACIÓN DE LOS FLUJOS DE DATOS

Descripción Técnica de la Arquitectura:

1. Capa de Captura y Adquisición Fija (Hardware e Ingesta):
   Los sensores (DHT11 y MQ-2) conectados al microcontrolador Arduino UNO toman lecturas periódicas de temperatura, humedad y gases ambientales. Estos datos son transmitidos mediante comunicación serie sobre USB hacia un script local en Python (HU-04a), el cual se encarga del pareo, validación y control de errores en la trama recibida.

2. Capa de Persistencia Relacional:
   El script de ingesta ejecuta sentencias INSERT parametrizadas para almacenar las lecturas estructuradas con marca temporal (TIMESTAMP) en la base de datos MySQL. En caso de pérdida de conectividad con el motor relacional, el sistema activa un buffer secundario en formato CSV (HU-04b) garantizando cero pérdida de datos históricos.

3. Capa de Servicios Backend y Lógica de Negocio:

   Se divide en dos componentes desacoplados para garantizar alta disponibilidad y escalabilidad:
   - Servicio API REST: se crea un pequeño servidor con Flask. El servidor consulta la base de datos MySQL y entrega los resultados en formato JSON para que otra parte de la aplicación — la página web— pueda mostrarlos. El archivo conecta la página con la base de datos y ofrece rutas para obtener la última medición, consultar históricos y generar una estimación sencilla.

   - Servicio Demonio de Alertas (Daemon Python): Atiende el Flujo Proactivo Asíncrono. Realiza un escaneo recurrente (polling cada 10 segundos) sobre el último registro en MySQL. Si identifica valores fuera de los rangos operativos (por ejemplo, temperaturas > 32º o < 8º, verifica el registro de cool-down (evitando duplicados en 30 minutos) e inicia la notificación.

4. Capa de Pasarela (Gateway) e Interfaz de Usuario:
   La pasarela API para Telegram actúa como puente seguro entre la infraestructura local backend y los dispositivos móviles. Transmite tanto las respuestas a las consultas directas como la emisión masiva de alertas críticas directamente hacia la comunidad educativa de la Escuela PRoA Río Tercero y los actores Municipales o Rurales.

5. Capa de Visualización:
   La interfaz desarrollada muestra un Dashboard Interactivo con todos los datos tomados de la Base de Datos principal en tiempo real. La ubicación del dispositivo mediante Geolocalización, donde se están tomando los datos de Temperatura, Humedad, Calidad del Aire y Probabilidad de Lluvia. También muestra un gráfico de lectura de datos, donde se muestra la variación de la Temperatura y de la Humedad durante las últimas 24 hs y también datos históricos de la variación de estas mismas variables pero de la última semana.
   Cuenta con dos botones mediante los cuales se generan reportes y muestras de los registros obtenidos, generando un documento PDF o Excel, los cuales pueden ser descargados y analizados por expertos, para una mejor toma de decisiones.
   Y cuenta también con un pronóstico calculado y simulado, del estado climático, para los próximos seis (6) días.

📊 Tabla de Stack Tecnológico:

Capa / Categoría	Componente Seleccionado	Función en el Sistema
Lenguaje Base	Python 3.12	Lenguaje principal para la lógica de ingesta serial, el servicio (Flask-API) y el demonio de alertas de fondo.

Backend Web / Webhook	Telegram API FlaskAPI	Framework REST asíncrono para la recepción de Webhooks HTTP POST de Telegram, el procesamiento de comandos y el retorno acelerado de respuestas. Flask-API, que es un microframework ligero para el lenguaje de programación Python.
Pasarela de Mensajería	Telegram API para Telegram	Gateway oficial de comunicación que desacopla la infraestructura local y permite el intercambio seguro de mensajes con los usuarios.
Base de Datos	MySQL (Server local)	Motor relacional encargado de la persistencia centralizada e historización de las series temporales capturadas por el hardware.
Librerías Clave	mysql-connector-python, pandas, colorama	Gestión de la conectividad SQL, análisis estadístico de promedios/tendencias térmicas y formateo visual de la consola local.
Despliegue y Orquestación	Docker & systemd	Contenedorización modular de la API Webhook y ejecución ininterrumpida del daemon Python de monitoreo como servicio del sistema (24/7).

03. JUSTIFICACIÓN DE DECISIONES TÉCNICAS Y DESCARTES

En concordancia con el requerimiento de la *Evidencia de Aprendizaje 1* de justificar brevemente el diseño e informar sobre alternativas descartadas, se documentan las siguientes decisiones clave:

1- Decisión Key: Desarrollo de una App Web o Móvil Propia elegido vs. Canal Telegram : Se seleccionó Telegram como la interfaz del usuario final por su nula fricción de adopción. Desarrollar una aplicación nativa o web de monitoreo meteorológico requiere que los usuarios descarguen una app o naveguen activamente a un sitio, generando bajas tasas de retención. Telegram, siendo una herramienta de uso cotidiano, asegura que tanto docentes, estudiantes como los productores locales de Río Tercero tengan acceso directo y natural a los reportes sin curva de aprendizaje.

2- Decisión Key: Flask vs. FastAPI para el Webhook: Se optó por Flask-API debido a su soporte nativo para programación síncrona (async/await), lo cual es crítico al manejar solicitudes Webhook de red concurrentes de la API de Telegram, logrando tiempos de respuesta de milisegundos y un consumo mínimo de hardware en la infraestructura local de la escuela. Se eligió Flask por su naturaleza síncrona por defecto. Flask API se utiliza para recibir peticiones HTTP (como `GET`, `POST`, `PUT`, `DELETE`) y devolver datos estructurados, generalmente en formato JSON, permitiendo la comunicación entre diferentes sistemas, aplicaciones móviles o servicios web.

3- Decisión Key: Daemon de Alertas en Python vs. Triggers/Disparadores en MySQL: Se determinó programar un servicio de fondo ('demonio') en Python independiente que escanee la base de datos MySQL en lugar de utilizar disparadores (triggers) SQL a nivel de motor. Un trigger en MySQL forzará al motor de base de datos a llamar a una función externa (UDF) para invocar APIs web externas (como Twilio), lo cual degrada severamente el rendimiento de la persistencia de datos y acopla de manera peligrosa la lógica de negocio a la base de datos. El demonio en Python mantiene la separación de responsabilidades intacta y facilita la gestión de reintentos e intervalos de gracia.

CONCLUSIÓN Y PRÓXIMOS PASOS DE CONSTRUCCIÓN

Esta propuesta arquitectónica y de requerimientos consolida un sistema inteligente, unificando el hardware existente en la Escuela PRoA Río Tercero con un flujo moderno de datos e interfaces de nula fricción. El chatbot de Telegram transforma los datos climáticos crudos en respuestas entendibles y alertas proactivas vitales para la comunidad educativa y local, cumpliendo íntegramente con la visión de **N2-EFMG Solutions**: *"Transformando el ruido de los sensores en decisiones inteligentes."*

01. Exploración y Justificación del Dato

¿De dónde sale el dato?

El dato se origina en el entorno físico de la Escuela PRoA Río Tercero mediante una arquitectura IoT local basada en Arduino UNO R3. Previo al armado físico, el circuito y la lógica de control fueron simulados y validados en la plataforma Tinkercad. La captura ambiental física se realiza a través de dos sensores principales:

- **Sensor DHT11:** Capta en tiempo real las variables de temperatura (en °C, con precisión entera) y humedad relativa (%).
- **Sensor MQ-2 (Gas / Humo):** Módulo electroquímico con cápsula metálica reticulada de color plateado y anillo identificador naranja, utilizado para detectar la presencia de gases combustibles (GLP, propano, metano), humo y vapores en el ambiente. Mide la concentración traduciendo la variación de resistencia del sensor a un valor analógico (analogRead(A0)) procesado por el Arduino UNO y convertido a Partes Por Millón (PPM) estimadas en la base de datos estacionmeteorologica_proa.

El microcontrolador (programado en C++ mediante Arduino IDE) empaqueta las lecturas en una trama de texto delimitada por comas (Humedad,Temperatura,Gas\r\n) y la transmite cada 2 segundos a 9600 baudios vía USB (puerto COM) hacia la computadora central de ingesta.

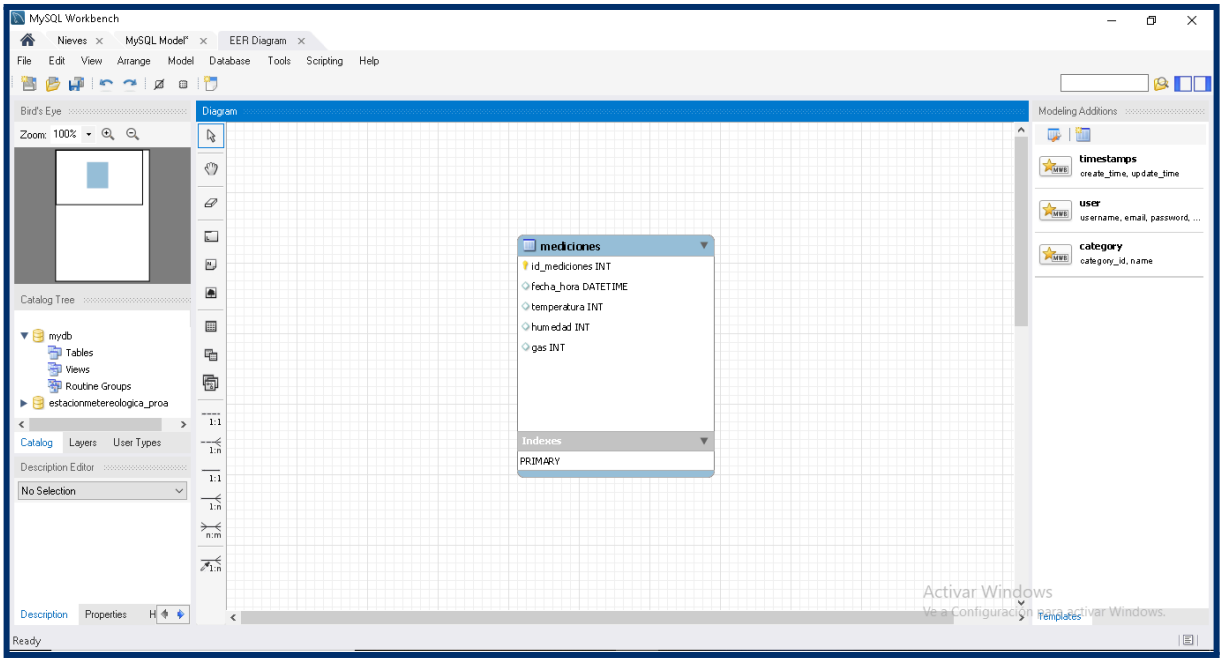
\r (Carriage Return / Retorno de Carro): Le indica al receptor que mueva el cursor al inicio de la línea.

\n (Line Feed / Salto de Línea): Le indica al receptor que avance una línea hacia abajo.

¿Qué base de datos, planilla o fuente usaron?

Para la captura y persistencia de la información, el sistema toma como fuente de origen una transmisión en vivo mediante el puerto de serie físico USB (COM), gestionado en Python 3.12 con la librería pyserial dentro de Visual

Studio Code. Desde allí, el flujo de datos se canaliza hacia una base de datos relacional MySQL Server administrada a través de MySQL Workbench, donde las lecturas se insertan de forma estructurada en la tabla mediciones del esquema estacionmeteorologica_proa.



¿Cómo está organizado y qué decidieron limpiar, descartar o transformar?

A. El dato fue estructurado relacionalmente en MySQL Workbench para optimizar espacio. La tabla mediciones contiene los siguientes campos:

- id_mediciones: Llave primaria autoincremental (INT AUTO_INCREMENT PRIMARY KEY).
- fecha_hora: Marca temporal automática generada por el motor relacional (DATETIME DEFAULT CURRENT_TIMESTAMP).
- temperatura: Guardado como entero (INT) debido a que el sensor DHT11 reporta variaciones de 1 en 1 grado.
- humedad: Guardado como entero (INT) para ocupar el mínimo espacio posible en disco.
- gas: Incorporado mediante la sentencia ALTER TABLE mediciones ADD COLUMN gas INT AFTER humedad para almacenar la concentración analógica del sensor MQ-2 en valores enteros (PPM).

B. Para cumplir con los criterios de la **HU-03 (Verificación de Comunicación Serial)**, en el entorno de Visual Studio Code (Python) se programó una canalización defensiva:

1. **Decodificación de Bytes Crudos:** El puerto COM transmite bytes (b'57,24,120\r\n'). Se aplica .decode('utf-8', errors='ignore').strip() para remover saltos de línea, retornos de carro y prevenir que el script falle ante caracteres basura al conectar el USB.
2. **Filtro de Descarte de Tramas Corruptas:** Si el sensor DHT11 falla o se desconecta un cable Dupont, el Arduino transmite "ERROR,ERROR,ERROR". El script Python evalúa if "ERROR" not in linea: y valida que la trama tenga exactamente 3 elementos (len(datos) == 3). Si la lectura llega incompleta o nula por ruido eléctrico, es descartada en memoria volátil, evitando inyectar errores dentro de MySQL.

3. **Parseo y Asignación Categórica de Gas:** Dado que el sensor MQ-2 es un dispositivo analógico de espectro amplio que no distingue la firma química exacta del gas, la clasificación del riesgo se resuelve en la capa de software (Python/SQL) modelando la severidad según rangos de PPM:
    - < 250 PPM: *Aire Normal / Estado Óptimo*.
    - 250 - 450 PPM: *Alerta Media (Humo o Vapores Suaves)*.
    - > 450 PPM: *Alerta Crítica (Fuga de Gas / Alta Concentración)*.
4. **Transformación Visual e Indicadores en Consola:** Se mapean los tipos a int y se renderizan marcos interactivos utilizando la librería colorama (mostrando en verde condiciones óptimas, en amarillo alertas medias y en rojo alertas críticas de gas o temperatura).
5. **Control de Flujo de Red:** Se asegura la reconexión e impresión continua del estado [CONECTADO] / [DESCONECTADO]. Mediante la propiedad autocommit=True en el conector de MySQL, se garantiza la inserción inmediata del registro sin bloqueos de transacciones.

🔹 ¿Por qué eligieron trabajarlo de esa manera?

Elegimos esta forma de tratarlos porque:

- **Calidad de Datos en la Cadena de Ingesta:** Preferimos filtrar las tramas defectuosas en el buffer de Python (antes de ejecutar la sentencia INSERT) evita contaminar las series temporales de la base de datos de la escuela.
- **Separación de Responsabilidades (Hardware vs. Software):** Al Arduino lo dejamos que se encargue únicamente de la lectura/actuación física (pantalla LCD, LEDs y Buzzer) y delegar a Python/MySQL la clasificación por software de los tipos de aire y alertas, optimiza el procesamiento del microcontrolador y da flexibilidad para ajustar umbrales sin reflashar el Arduino.
- **Eficiencia de Almacenamiento:** Guardar variables numéricas completas como INT optimiza el tamaño de la base de datos en MySQL Workbench, permitiendo soportar lecturas continuas sin sobrecargar los recursos informáticos del colegio.
- **Validación Visual de QA:** La representación visual con colores en la consola interactiva de Python permite a docentes y alumnos validar el acoplamiento del hardware previo a la etapa de consumo vía WhatsApp.

02. HU-AGRO-01: Persistencia Automática y Visualización Online de Datos Climáticos Rurales

Con el fin de dar continuidad al proyecto atendiendo las necesidades de los trabajadores del sector Rural del éjido municipal de Río Tercero y alrededores, elaboramos la historia de usuario correspondiente, siguiendo los lineamientos respecto a los criterios de aceptación y el método I.V.E.S.T aplicables a los pain point del Usuario propiamente dicho.

🔹 1. Historia de Usuario

- **Como** productor agrícola de la zona de Río Tercero,
- **quiero** acceder a una plataforma en línea que registre automáticamente los datos en tiempo real de la estación meteorológica y me permita consultar gráficos históricos por rangos de tiempo,
- **para** disponer de información precisa del microclima local y tomar decisiones fundamentadas de siembra, riego y prevención, evitando pérdidas económicas derivadas de depender de estaciones distantes.

🔹 2. Criterios de Aceptación

- **Escenario 1: Almacenamiento e Ingesta Automática en Tiempo Real**
  - **Dado** que la estación meteorológica está activa registrando datos en Río Tercero,
  - **Cuando** el sistema recibe una nueva lectura del microcontrolador,
  - **Entonces** el registro se inserta automáticamente en la base de datos MySQL con su marca de tiempo (timestamp) precisa y sin intervención manual.

- **Escenario 2: Visualización de Tendencias Climáticas en la Página Web**
  - **Dado** que existen registros meteorológicos acumulados en la base de datos,
  - **Cuando** el productor agrícola ingresa al sitio web y selecciona un intervalo de tiempo (ej. últimas 24 horas, última semana o rango de fechas personalizado),
  - **Entonces** la página renderiza un gráfico interactivo (de líneas o barras) con ejes etiquetados (Tiempo vs. Temperatura/Variables climáticas) para identificar picos y patrones térmicos locales.

- **Escenario 3: Resiliencia del Almacenamiento en Zona Rural**
  - **Dado** que la conexión a la base de datos central o a internet se interrumpe en el sector rural,
  - **Cuando** el sensor continúa capturando lecturas,
  - **Entonces** el sistema resguarda los datos localmente en un archivo de buffer y los sincroniza automáticamente con la plataforma web al restablecerse la red.

🔹 3. Evaluación bajo el Criterio INVEST

- **I – Independiente:** Es autónoma dentro de la arquitectura web al acoplar el proceso de captura en segundo plano con la vista de consulta en línea.
- **N – Negociable:** Permite negociar los tipos de gráficos, los parámetros visualizados (temperatura, humedad o lluvias) y la frecuencia de actualización del portal. Siempre que se dé prioridad a las visualizaciones de los datos más relevantes para el productor.
- **V – Valiosa:** Resuelve directamente el *pain point* agrícola de Río Tercero al eliminar la incertidumbre sobre microclimas locales y optimizar la toma de decisiones productivas.
- **E – Estimable:** Combina una tarea de persistencia directa en MySQL con el renderizado web mediante librerías de visualización conocidas. Lo que llevaría un desarrollo aproximado de entre 25 a 35 líneas de código.
- **S – Small (Adecuada):** Delimita el almacenamiento automático y la consulta visual dentro de una misma interfaz de usuario. El desarrollo puede realizarse en 2 semanas aproximadamente.
- **T – Testeable:** Se valida mediante pruebas de inserción continua en la base de datos y la verificación del gráfico resultante en el navegador.

❖ Acceso a pruebas y videos documentados de la aplicación en funcionamiento:

- Videos y Photos de PRUEBAS

03. Informe Breve: Actualización de la Justificación de la Evidencia 1

En la Evidencia 1 se planteó como problema central, la falta de una infraestructura centralizada y automatizada para almacenar e historiar las lecturas de la Estación Meteorológica y Ambiental de la Escuela PRoA Sede Río Tercero, lo que provocaba la pérdida de datos al desconectar el hardware y la imposibilidad de realizar análisis estadísticos. La propuesta original justificó la creación de una canalización de ingesta en Python para persistir las variables en MySQL (respaldada por un buffer en CSV) y el desarrollo de un Chatbot de WhatsApp como interfaz de baja fricción para democratizar las consultas de la comunidad educativa y municipal.

En esta segunda etapa, la justificación se fortalece y expande, se confirmó que la captura física vía USB y el procesamiento desacoplado garantizan la integridad de los datos, y se amplió el impacto de la solución para responder también a las necesidades operativas de los productores agrícolas del ejido rural de Río Tercero, integrando visualizaciones de tendencias en tiempo real e historiales climáticos.

Lo que se sostiene igual:

- **El problema de origen:** La necesidad de eliminar el registro manual desde el Monitor Serie y evitar la pérdida de las series temporales de la estación ambiental.
- **La arquitectura base de ingesta y persistencia:** El uso del microcontrolador Arduino UNO R3 enviando tramas por puerto serie USB hacia un script local en Python 3.12, que persiste los registros de forma estructurada en la base de datos relacional MySQL (estacionmeteorologica_proa), utilizando un buffer local en formato CSV como mecanismo de resiliencia ante caídas de conexión.
- **Las decisiones de diseño clave:** La separación estricta de responsabilidades entre el hardware (dedicado a lectura y actuación física) y el software (Python/MySQL para lógica y alertas), así como la decisión de evitar *triggers* a nivel de base de datos para no degradar el rendimiento del motor relacional ni acoplar llamadas HTTP externas.

Lo que cambió o evolucionó:

- **Integración y tipificación del sensor MQ-2 (Gas/Humo):** Mientras que en la fase inicial el enfoque fue primordialmente térmico y de humedad, en la Evidencia 2 se consolidó la captura del sensor analógico MQ-2, modificando la estructura de la tabla mediciones (ALTER TABLE mediciones ADD COLUMN gas INT AFTER humedad) e incorporando la clasificación categórica del nivel de riesgo en PPM por software.

- **Incorporación del usuario agrícola (HU-AGRO-01):** Se identificó y formalizó la necesidad del sector rural circundante de acceder a una plataforma en línea con gráficos interactivos de tendencias históricas (temperatura, humedad, gases) para fundamentar decisiones de siembra y riego.
- **Mecanismos defensivos en el código:** Se profundizó la canalización defensiva en Python agregando filtros automáticos de descarte para tramas corruptas ("ERROR" o longitud distinta a 3 elementos) y validación visual con la librería colorama en consola antes de la integración con Telegram.
- **Migración de la interfaz de interacción:** Se redefinió el canal de consulta para la comunidad educativa y los productores agrícolas, reemplazando la propuesta inicial de WhatsApp por Telegram. Esta decisión se fundamenta en que Telegram es un ecosistema libre y completamente gratuito, lo que elimina la dependencia de intermediarios de pago (como las API comerciales de WhatsApp Business / Twilio) y garantiza la sustentabilidad económica del proyecto a largo plazo para la institución. Además, la Telegram Bot API ofrece capacidades superiores para el desarrollo de chatbots: permite gestionar comandos nativos estructurados (como /clima, /estado o /reporte), integrar teclados interactivos (Inline Keyboards), procesar eventos mediante Webhooks de manera directa y enviar alertas o gráficos generados por la estación con menor latencia y sin restricciones operativas.

Aprendizajes al trabajar el dato:

- **Ruido eléctrico y fallas de hardware:** Se constató que la señal serie física sufre interferencias o desconexiones (ej. cables Dupont sueltos), por lo que es indispensable filtrar y limpiar los bytes crudos en la memoria volátil de Python antes de ejecutar sentencias INSERT para evitar contaminar la base de datos de la escuela.
- **Manejo de sensores analógicos de espectro amplio:** Aprendimos que el sensor MQ-2 no identifica la firma química exacta del gas; por ende, la severidad del riesgo debe modelarse en software mediante rangos de PPM (<250 Aire Normal, 250-450 Alerta Media, >450 Alerta Crítica) sin recargar ni re-flashear el microcontrolador Arduino.
- **Optimización de espacio:** Confirmamos que almacenar las variables numéricas como enteros (INT) en MySQL optimiza el almacenamiento en disco para soportar registros continuos cada 2 segundos sin sobrecargar la infraestructura del colegio.

Aprendizajes al construir la historia de usuario (HU-AGRO-01):

- **Resiliencia en entornos rurales:** Al evaluar la historia bajo el criterio INVEST, comprendimos que la tolerancia a fallos de conectividad (buffer local y sincronización automática) es una condición funcional indispensable para que la solución sea verdaderamente valiosa en el campo.
- **Granularidad e Independencia:** Validamos que la captura continua en segundo plano y la consulta visual web pueden empaquetarse de manera autónoma, permitiendo negociar el tipo de gráficos e indicadores sin bloquear el desarrollo de otros módulos.

🔹 Ajustan de cara a lo que sigue

- **Despliegue e integración final del Chatbot (Telegram Bot API + FastAPI):** Transicionar de la lectura y validación visual local en consola (colorama) a la puesta en producción de un servidor Webhook desacoplado mediante FastAPI e integrado con la Telegram Bot API. Esto permitirá procesar en tiempo real los comandos nativos de Telegram (/clima, /estado, /reporte) e interactuar de forma fluida con la comunidad educativa y los productores rurales.
- **Orquestación y disponibilidad continua (24/7):** Configurar el servicio demonio de alertas proactivas (para temperaturas >32°C / <8°C y niveles críticos de gas) como un proceso persistente systemd e implementar la contenedorización con Docker para asegurar un uptime objetivo del 99.5%.
- **Agregaciones estadísticas avanzadas:** Utilizar Pandas en el backend para automatizar las agregaciones diarias de máximas, mínimas y cálculo de la pendiente de temperatura de la última hora sobre MySQL, alimentando tanto las respuestas del bot de Telegram como la interfaz web de tendencias.